Computer Vision Concepts — Zebra Interview Reference

Oct 27, 2026 · @Nishal

Everything below is what a Computer Vision Software Developer at Zebra (ex-Matrox Imaging) is expected to explain in under a minute. Each concept: what it is, the key formula or steps, and where it breaks. Items marked **[Aurora]** map to Aurora Imaging Library tools.

1. Image fundamentals

- **Digital image:** a 2D array of samples. Memory layout is row-major with a **stride (pitch)** of at least width × bytes per pixel, padded for alignment, so always index `y * stride + x`. Bit depths: 8-bit (0–255), 10/12/16-bit sensors, 32-bit float for intermediate results.
- **Interleaved vs planar:** RGBRGB… versus separate R, G and B planes. Planar data vectorizes better per channel.
- **Bayer pattern:** a single sensor with an RGGB colour filter array. Demosaicing interpolates the missing channels and causes zipper artifacts and false colour at edges. Industrial monochrome cameras avoid it: sharper images and more light per pixel.
- **Colour spaces:** RGB is device-dependent. **HSV/HSL** separate hue from brightness, which makes colour thresholds robust to shading. **Lab** is perceptually uniform, so Euclidean distance approximates perceived difference (ΔE), and it's used for colour inspection. **YCbCr/YUV** separate luma from chroma; video codecs subsample chroma (4:2:0) because the eye is less sensitive to colour detail. Grey from RGB: Y = 0.299R + 0.587G + 0.114B (BT.601).
- **Sampling and aliasing:** Nyquist: you must sample at more than twice the highest spatial frequency. Downsampling without low-pass filtering first causes aliasing (moiré), so blur before decimating; that's why image pyramids use a Gaussian.
- **Quantization:** bit depth limits dynamic range. Gamma: sRGB values are not linear in light, so convert to linear space before photometric maths.
- **Noise models:** shot noise (Poisson, grows with $\sqrt{\text{signal}}$), read noise (Gaussian), salt-and-pepper (impulse, dead or hot pixels), fixed-pattern noise (fix with flat-field correction). The usual remedies: Gaussian blur for Gaussian noise, median filter for impulse noise, frame averaging for static scenes.

- **Image formation and optics:** exposure = aperture × shutter × gain. Depth of field grows as the aperture shrinks (higher f-number). Motion blur ≈ speed × exposure time; keep it under 1 pixel by using a short exposure and strobe lighting. **Telecentric lenses** have constant magnification with distance, so they're used for gauging.
- **Lighting (half of industrial vision):** backlight for silhouettes and gauging; diffuse dome light for shiny or curved parts; low-angle dark-field light to highlight scratches and embossing; coaxial light for flat reflective surfaces; structured light for 3D. Polarizers cut glare.
- **Resolution planning:** pixels needed = field of view ÷ smallest feature × (3–5 pixels per feature for detection, more for measurement). For example, a 100 mm field of view with 0.1 mm defects needs about 3,000–5,000 pixels across.

2. Filtering and enhancement

- **Convolution vs correlation:** convolution flips the kernel; correlation doesn't. They are identical for symmetric kernels. Cost is $O(k^2)$ per pixel for a k×k kernel.

```
g(x,y) = \sum_{i=-r}^{r} \sum_{j=-r}^{r} h(i,j)\, f(x-i,\, y-j)
```

- **Separability:** a kernel that is an outer product (Gaussian, box, Sobel) splits into a horizontal pass and a vertical pass, so cost drops from $O(k^2)$ to $O(2k)$. Test: the kernel matrix has rank 1.
- **Border handling:** constant (zero), replicate, reflect, or wrap. Always say which one you use; reflect is the usual default.
- **Box (mean) filter:** O(1) per pixel with an integral image or a running sum. Its frequency response has ringing; three box passes approximate a Gaussian.
- **Gaussian filter:** isotropic and separable, choose kernel radius ≈ 3σ. It's the only kernel that doesn't create new structure as you blur (scale-space). Large σ: use a pyramid or recursive (IIR, Deriche) filters.
- **Median filter:** nonlinear; removes salt-and-pepper noise while keeping edges. Fast versions use histogram methods: O(1) per pixel for 8-bit images (Perreault).
- **Bilateral filter:** weights by both spatial distance and intensity difference, so it smooths while preserving edges. It's slow; approximations exist (guided filter). **Non-local means:** averages similar patches.
- **Sharpening:** unsharp mask: out = in + α · (in − blur(in)). The Laplacian (second derivative) amplifies noise. **LoG / DoG** are blob and edge detectors; DoG approximates LoG and is what SIFT uses.
- **Point operations:** brightness/contrast (gain and offset), gamma, look-up tables (LUTs). A LUT is the fastest way to apply any per-pixel 8-bit mapping: 256 entries.

- **Histogram:** the distribution of intensities. **Equalization** maps through the normalized CDF to spread contrast. **CLAHE** equalizes in tiles with a clip limit so noise isn't over-amplified. **Histogram matching** maps one image's histogram onto another's.
- **Frequency domain:** the FFT turns convolution into multiplication, so it's faster for large kernels (roughly above 15×15). Low-pass = blur, high-pass = edges, band-stop removes periodic patterns (for example, fabric weave or screen moiré). Phase correlation (normalized cross-power spectrum) estimates translation between images to sub-pixel accuracy.
- **Flat-field correction:** corrected = (raw − dark) / (flat − dark) × mean. It removes vignetting and pixel-gain differences, and is standard before measurement.
- **Image arithmetic:** subtracting a golden template for defect detection requires alignment first and a tolerance band for small shifts. Use saturating arithmetic so values don't wrap.

3. Edges, corners and features

- **Image gradient:** $\nabla I = (I_x, I_y)$; magnitude = $\sqrt{(I_x^2 + I_y^2)}$ (or $|I_x| + |I_y|$ for speed); direction = atan2(I_y, I_x). **Sobel** = smoothing × derivative, separable as $[1\ 2\ 1]^T \times [-1\ 0\ 1]$. **Scharr** is more rotationally accurate. **Prewitt** and **Roberts** are older and noisier.
- **Canny edge detector (likely written-test question):**
  1. Gaussian smoothing (σ sets the scale).
  2. Gradient magnitude and direction (Sobel).
  3. Non-maximum suppression: keep a pixel only if it is the maximum along the gradient direction, so edges are 1 pixel wide.
  4. Double threshold: strong (> high), weak (between low and high), dropped (< low). A typical high:low ratio is 2:1 to 3:1.
  5. Hysteresis: keep weak pixels only when connected to strong ones.
- Canny was designed for good detection, good localization and a single response per edge. Its weaknesses: corners get rounded, and it depends on thresholds. **Canny–Deriche** uses recursive (IIR) filters, so speed doesn't depend on σ [Aurora edge tools].
- **Sub-pixel edges:** fit a parabola to 3 gradient-magnitude samples across the edge, or find the zero crossing of the second derivative. Gives about 1/10–1/50 pixel accuracy with good optics.
- **Harris corner detector:** structure tensor $M = \Sigma\, w\, [I_x^2, I_xI_y; I_xI_y, I_y^2]$; response $R = \det(M) - k \cdot \mathrm{trace}(M)^2$ with $k \approx 0.04$–0.06. Two large eigenvalues = corner, one = edge, none = flat. **Shi–Tomasi** uses $\min(\lambda_1, \lambda_2)$ and is better for tracking. **FAST** is a segment test on a 16-pixel circle: very fast, no orientation.

- **Blob detection:** LoG/DoG across scales finds extrema in (x, y, σ). MSER finds stable regions across thresholds, which is good for text.
- **Descriptors:**
  - **SIFT:** DoG keypoints, dominant orientation, 4×4 grid of 8-bin gradient histograms = 128-D float vector. Invariant to scale and rotation, partly to lighting. Patent expired in 2020.
  - **SURF:** box-filter approximation, faster.
  - **ORB:** FAST + oriented BRIEF, a 256-bit binary descriptor matched by Hamming distance (XOR + popcount). Fast and free; the usual choice for real-time.
  - **AKAZE:** nonlinear scale space, binary descriptor, good accuracy for its speed.
  - **HOG:** gradient-orientation histograms on a cell grid, the classic for pedestrian detection with an SVM.
- **Matching:** brute force or approximate nearest neighbour (FLANN, k-d trees, LSH for binary descriptors). **Lowe's ratio test:** accept if d1 / d2 < 0.7–0.8. Cross-check (mutual nearest neighbour).
- **RANSAC:** repeatedly sample a minimal set (2 points for a line, 4 for a homography, 8 or 7 for F, 5 for E), fit, count inliers within a threshold, keep the best, then refit on all inliers. Iterations $N = \log(1 - p) / \log(1 - w^s)$, with w = inlier ratio and s = sample size; for example, w = 0.5, s = 4, p = 0.99 gives about 72 iterations. Variants: MSAC, PROSAC, LO-RANSAC, MAGSAC.
- **Learned features:** SuperPoint + SuperGlue / LightGlue, and LoFTR (detector-free). They're more robust under viewpoint and lighting change, but need a GPU.

4. Segmentation and morphology

- **Global threshold:** fixed, or **Otsu**, which picks the t that maximizes between-class variance $\omega_0\omega_1(\mu_0 - \mu_1)^2$. One pass over the 256-bin histogram with running sums. Assumes a bimodal histogram.
- **Adaptive (local) threshold:** compare each pixel with its local mean or Gaussian mean minus C; Sauvola/Niblack for documents. Handles uneven lighting. Fast with integral images.
- **Morphology** (binary or greyscale, with a structuring element):
  - Erosion = local minimum: shrinks objects, removes specks.
  - Dilation = local maximum: grows objects, fills gaps.
  - **Opening** = erode then dilate: removes small bright noise while keeping shape.
  - **Closing** = dilate then erode: fills small holes and gaps.
  - **Top-hat** = image − opening: small bright features on an uneven background. **Black-hat** = closing − image: small dark features.

    - Morphological gradient = dilate − erode, giving an outline. Hit-or-miss finds shape patterns. Skeletonization / thinning gives the medial axis.
    - A rectangular element is separable (row then column), and the van Herk/Gil–Werman algorithm makes it O(1) per pixel.
- **Connected components labelling:** two-pass with union–find (label, record equivalences, resolve), or BFS/DFS flood fill (use an explicit stack to avoid recursion overflow). 4- vs 8-connectivity: use opposite connectivities for foreground and background to avoid paradoxes.
- **Blob analysis [Aurora]:** per blob: area, perimeter, centroid, bounding box, moments, orientation (from second-order central moments), eccentricity, compactness = $4\pi A/P^2$, convexity, holes (Euler number). Filter blobs by these features: the classic classical defect classifier.
- **Image moments:** $m_{pq} = \Sigma\, x^p\, y^q\, I(x,y)$; centroid = (m10/m00, m01/m00); orientation $\theta = \tfrac{1}{2}\,\mathrm{atan2}(2\mu_{11}, \mu_{20} - \mu_{02})$. **Hu moments** are invariant to translation, scale and rotation.
- **Contours:** border following (Suzuki), polygon simplification (Douglas–Peucker), convex hull and convexity defects, minimum-area rectangle (rotating calipers), and ellipse fitting.
- **Region growing / split-and-merge:** grow from seeds by a similarity criterion.
- **Watershed:** treat the gradient image as a landscape and flood from markers; lines where floods meet become boundaries. Marker-controlled watershed (markers from a distance transform) separates touching objects such as coins or cells.
- **Distance transform:** distance of each pixel to the nearest background pixel. Chamfer approximation in 2 passes, or exact Euclidean (Felzenszwalb). Uses: separating touching blobs, measuring thickness, chamfer matching.
- **Graph-based / energy methods:** GrabCut (graph cuts + GMM colour models), active contours (snakes), level sets. Clustering: k-means on colour, mean-shift, SLIC superpixels.
- **Deep segmentation:** see section 10 (U-Net, Mask R-CNN, SAM).

5. Pattern matching and measurement

- **Template matching scores:**
    - SAD/SSD (sum of absolute or squared differences): fast, but sensitive to brightness changes.
    - **NCC (normalized cross-correlation)**, below: invariant to linear gain and offset; range −1 to 1; undefined on flat windows (zero variance).

```
\text{NCC}(u,v) = \frac{\sum (I - \bar{I}_{u,v})(T - \bar{T})}{\sqrt{\sum (I - \bar{I}_{u,v})^2 \, \sum (T - \bar{T})^2}}
```

- **Fast NCC:** integral images give each window's sum and sum of squares in O(1); the numerator via FFT for large templates; a coarse-to-fine **pyramid** search (search the whole image at low resolution, refine locally); early termination; restrict search to a region of interest. Rotation and scale need multiple templates or a rotation search, which costs time, so use the next method.
- **Geometric (edge-based) model finding [Aurora Model Finder]:** the model is a set of edge points with gradient directions, not pixel values. Search over position, angle and scale by scoring how well edge points line up (generalized Hough or chamfer-like matching). Robust to lighting changes, occlusion and clutter; returns sub-pixel pose and a match score. This is the core Matrox tool; know the trade-off against NCC (NCC is better for textured parts with weak edges).
- **Hough transform:** each edge point votes in a parameter space. Lines: $\rho = x\cos\theta + y\sin\theta$. Circles: (a, b, r), reduced by using the gradient direction. The probabilistic Hough transform is faster. The **generalized Hough transform** works for arbitrary shapes using an R-table.
- **Chamfer matching:** distance transform of the image's edges; score = mean distance under the template's edge points.
- **Sub-pixel peak location:** fit a parabola (or 2D quadratic) to the score around the best integer position: offset = $(s_{-1} - s_{+1}) / (2(s_{-1} - 2s_0 + s_{+1}))$.
- **Metrology / gauging [Aurora Metrology, Measurement]: caliper tools** project a 1D intensity profile across an edge, find edge pairs and measure width. Fit lines and circles to sub-pixel edge points with least squares or RANSAC. Geometric tolerances: distance, angle, parallelism, circularity. Convert pixels to millimetres through calibration.
- **Measurement repeatability:** report 3σ of repeated measurements of the same part. **Gauge R&R** separates repeatability (same operator) from reproducibility (different conditions). A common target: the system's error under 10% of the tolerance band.
- **Alignment before inspection:** find the part (model finder), compute the pose, then warp regions of interest into the part's coordinate frame (a "fixture") so the inspection tools stay in place when the part shifts.
- **Golden-template comparison:** register to a reference, subtract, threshold with a tolerance that is larger near edges (local variation), then blob-analyse what's left. Simple, but sensitive to normal part-to-part variation.

6. OCR and code reading

- **Industrial OCR vs document OCR:** industrial text is short strings (lot codes, expiry dates, serial numbers) printed by inkjet dot-matrix, laser marking or embossing, on curved or reflective surfaces with low contrast. **Verification** (OCV, confirming known text is correct and legible) is often more important than reading unknown text.
- **Classical OCR pipeline [Aurora String Reader]:**
  1. Locate the text region (model finder, or projection profiles).
  2. Preprocess: deskew (from projections or Hough), adjust contrast, binarize, close dot-matrix gaps with morphology.
  3. Segment characters: connected components, vertical projection profiles, or fixed pitch for known fonts.
  4. Normalize each character's size and position.
  5. Classify against a **font definition** (templates, or features plus nearest neighbour or SVM).
  6. Verify with syntax rules (for example, `[A-Z]{2}[0-9]{6}`), date validity and a checksum.
- **Font definition from samples:** extract the characters from a sample string with known text and average the instances into per-character models. This is Matrox's automatic font definition approach. A key problem is touching or broken characters.
- **Deep OCR:** a detector (EAST, DBNet, CRAFT) plus a recognizer such as **CRNN + CTC** (CNN features → BiLSTM → CTC loss, no per-character segmentation needed). Newer: attention or transformer recognizers (TrOCR, PARSeq). Train on synthetic text rendered in the target font with augmentation (blur, perspective, noise, lighting) plus a small real set.
- **CTC in one line:** it adds a blank symbol and sums over all alignments of the label to the per-frame outputs; decoding merges repeated characters and removes blanks.
- **1D barcodes (EAN/UPC, Code 128, Code 39):** sample a scanline, find the edges, convert bar and space widths to modules, decode, check the checksum. Robust readers try several scanlines and handle blur with sub-pixel edges. Quality grading follows ISO/IEC 15416.
- **2D codes (QR, Data Matrix, Aztec):** find the finder pattern (QR's three corner squares; Data Matrix's L-shaped edge and clocking track), correct perspective, sample the module grid, then **Reed–Solomon** error correction (QR levels L/M/Q/H recover about 7/15/25/30%). Grading: ISO/IEC 15415, and AIM DPM for direct part marks. Zebra's core business, so expect interest.
- **DPM (direct part marking):** dot-peened or laser-etched codes on metal, with low contrast and specular surfaces. Fix with dome or low-angle lighting, morphology to merge dots, and a reader tolerant of uneven module sizes.

- **Your evidence:** multilingual care-label verification, tech-pack OCR with layout segmentation, and reading digits from medical monitors (seven-segment displays).

7. Geometry and calibration

- **Homogeneous coordinates:** a 2D point (x, y) becomes (x, y, 1), and points are equal up to scale. They make translation and projection linear (matrix multiplication) and represent points at infinity.
- **2D transform hierarchy (degrees of freedom):** translation (2) → Euclidean/rigid (3) → similarity (4: + scale) → affine (6: parallel lines stay parallel) → projective/homography (8: straight lines stay straight). Minimum point pairs = DOF ÷ 2: 1, 2, 2, 3, 4.
- **Pinhole camera model:**

```
s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \, [\,R \mid t\,]
\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \qquad K = \begin{bmatrix}
f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}
```

- **Intrinsics (K):** focal lengths in pixels, principal point, skew (≈ 0). **Extrinsics (R, t):** camera pose in the world, 6 DOF. Focal length in pixels = focal length in mm ÷ pixel pitch in mm.
- **Lens distortion (Brown–Conrady):** radial $x_d = x(1 + k1r^2 + k2r^4 + k3r^6)$ (barrel or pincushion) plus tangential (p1, p2) from lens decentring. Fisheye lenses need a different model (equidistant). Undistort with a precomputed remap table, computed once and then applied per frame.
- **Calibration (Zhang's method):** image a planar target (checkerboard, circle grid, ChArUco) in 10–20 poses, find corners to sub-pixel precision, estimate a homography per view, solve for K in closed form, then refine all parameters with **Levenberg–Marquardt** to minimize reprojection error. A good result is RMS < 0.3–0.5 px. Cover the whole field of view and tilt the target. ChArUco works with partial views.
- **Pixel-to-world calibration for 2D inspection [Aurora calibration]:** a calibration grid on the inspection plane gives a mapping, including perspective and distortion, from pixels to millimetres. Then all measurements are in real units.
- **Homography (planar mapping):** $x' \sim Hx$, 3×3, 8 DOF. Estimate it with the **DLT** (stack 2 equations per point pair, solve $Ah = 0$ by SVD, taking the last right singular vector) after **normalizing** the points (Hartley: centroid at the origin, mean distance $\sqrt{2}$), then use RANSAC for outliers and refine nonlinearly. Uses: perspective correction of a plane, mosaics, homography between a planar target and the image.
- **Epipolar geometry (two views):** a point in one image constrains its match to a line in the other.

  - Fundamental matrix F: $x'^T F x = 0$, rank 2, 7 DOF, estimated with the **8-point algorithm** (normalized) or the 7-point algorithm + RANSAC. Uncalibrated.
  - Essential matrix $E = K'^T F K = [t]_\times R$, 5 DOF; decompose by SVD into 4 (R, t) solutions and pick the one with points in front of both cameras (cheirality check). Translation is known only up to scale.
- **Triangulation:** intersect two rays (linear DLT, or the midpoint method), then minimize reprojection error. Accuracy drops as the baseline shrinks or the angle between rays gets small.
- **PnP (Perspective-n-Point):** camera pose from known 3D–2D correspondences. P3P minimal solver + RANSAC, then iterative refinement. Uses: robot guidance, locating a part in 3D from a CAD model.
- **Hand–eye calibration:** solve $AX = XB$ for the transform between camera and robot flange (Tsai–Lenz). Relevant to vision-guided robotics such as Photoneo's.
- **Bundle adjustment:** jointly refine all camera poses and 3D points by minimizing total reprojection error (sparse Levenberg–Marquardt, Schur complement; Ceres or g2o). It's the final step of SfM and SLAM.

8. 3D vision

Method	Principle	Strengths	Weaknesses
Passive stereo	Two calibrated cameras; rectify, match, depth $Z = fB/d$	No projector; works outdoors	Fails on textureless surfaces; matching is costly
Active stereo	Stereo + projected random speckle to add texture	Works on plain surfaces	Speckle washes out in sunlight
Structured light	Project known patterns (Gray code + phase shift) and decode the correspondence for each pixel	High accuracy and density; Photoneo's domain	Needs a static scene per capture; specular problems
Laser triangulation (sheet of light)	Laser line + angled camera; scan by moving the part	Very precise profiles; fast line rates	One profile per frame; occlusion (shadowing)
Photometric stereo	Fixed camera, 3+ lighting directions → surface normals	Fine surface detail (scratches, embossing)	Relative shape only; assumes Lambertian surfaces

Method	Principle	Strengths	Weaknesses
Time of flight (ToF)	Phase or pulse travel time of light	Compact, real-time, full frame	Lower resolution; multipath; flying pixels at edges

- **Stereo pipeline:** calibrate both cameras → **rectify** (epipolar lines become horizontal rows) → match along rows to get a **disparity** map → depth $Z = f \cdot B / d$. Depth error grows as $Z^2 / (f \cdot B)$ per pixel of disparity error, so precision falls off quadratically with distance. Matching: block matching (SAD, census transform), **SGM** (semi-global matching, the industry standard), or learned methods (RAFT-Stereo). Validate with a left–right consistency check.
- **Structured light decoding: Gray code** (binary stripes where neighbours differ by one bit) gives a coarse absolute projector column. **Phase shifting** (N sinusoids shifted by $2\pi/N$): $\varphi = \text{atan2}(\Sigma I_n \sin \delta_n, \Sigma I_n \cos \delta_n)$ gives a fine but wrapped phase, which is **unwrapped** with the Gray code. Then triangulate the camera ray against the projector plane. Photoneo's "parallel structured light" captures it in one shot using a mosaic sensor, which handles moving scenes.
- **Laser triangulation details:** find the line peak in each column at sub-pixel precision (centre of gravity, or Gaussian fit), intersect the camera ray with the calibrated laser plane, and use the encoder to step along the scan. Problems: laser speckle, reflections from shiny parts (multiple peaks: choose the strongest or the first), and camera or laser occlusion (fix with two cameras or two lasers).
- **Photometric stereo:** $I = \rho (n \cdot l)$ for each light; with 3+ lights, solve per pixel by least squares for ρn, then normalize. Integrate the normal field to get height (Frankot–Chellappa in the Fourier domain, or Poisson). Reject shadowed and specular pixels as outliers. Great for reading embossed text and spotting dents.
- **Point clouds:** an unordered set of XYZ points (+ normals or colour). A depth map is an organized cloud. Operations:
  - Voxel-grid downsampling; statistical outlier removal.
  - Normal estimation: PCA of k nearest neighbours; the smallest eigenvector is the normal.
  - Plane fitting with RANSAC (remove the conveyor or table surface), then cluster what remains (Euclidean clustering) to separate objects.
  - **ICP** registration: alternate between finding closest-point correspondences and solving the rigid transform (SVD, Kabsch). Needs a good initial guess; point-to-plane ICP converges faster. Global initialization: FPFH features + RANSAC.
  - k-d tree or octree for nearest-neighbour search. Libraries: PCL, Open3D.

- **3D inspection:** compare a scan with CAD or a golden part (deviation colour map), measure height and volume, check flatness and coplanarity (for example, connector pins), and pick parts from a bin (pose estimation → grasp → robot).

9. Motion and tracking

- **Brightness constancy:** $I(x, y, t) = I(x + u, y + v, t + 1)$ linearizes to the **optical flow constraint** $I_x \cdot u + I_y \cdot v + I_t = 0$: one equation, two unknowns (the **aperture problem**).
- **Lucas–Kanade:** assume constant flow in a small window and solve by least squares with the same structure tensor as Harris. That's why corners track well and edges don't. Use a pyramid for large motions. **KLT tracker** = Shi–Tomasi corners + pyramidal LK.
- **Horn–Schunck:** dense flow with a global smoothness term. Modern dense flow: Farnebäck (polynomial expansion), deep models (RAFT).
- **Background subtraction:** running average, or a mixture of Gaussians per pixel (MOG2). Handles slow lighting changes; shadows need special handling.
- **Kalman filter:** linear-Gaussian state estimator. Predict $\hat{x} = Fx$, $P = FPF^T + Q$; update with gain $K = PH^T(HPH^T + R)^{-1}$. Constant-velocity model for tracking a box or point. EKF/UKF for nonlinear models; particle filters for multi-modal distributions.
- **Multi-object tracking:** detect per frame, then associate detections with tracks using the **Hungarian algorithm** on an IoU or distance cost, with a Kalman filter for prediction (SORT). DeepSORT adds appearance embeddings; ByteTrack also uses low-confidence detections. Metrics: MOTA, IDF1, ID switches.
- **Conveyor tracking (industrial):** an encoder gives the belt position, so a part detected at one camera can be predicted at the reject station: latency budget = distance ÷ belt speed.
- **Visual odometry / SLAM:** track features → estimate relative pose (essential matrix or PnP) → triangulate landmarks → local bundle adjustment → loop-closure detection (bag of words) → pose-graph optimization. ORB-SLAM is the reference system. Visual-inertial (VIO) adds an IMU for scale and robustness.
- **Structure from Motion (SfM):** unordered photos → feature matching → incremental reconstruction (two-view initialization, then PnP for each new image) → bundle adjustment; COLMAP. Multi-view stereo (MVS) then produces a dense reconstruction. Be honest that you know the pipeline but haven't built a SLAM system.

10. Deep learning for vision

- **CNN building blocks:** convolution layer (weight sharing, local receptive field); output size = ⌊(W − K + 2P) / S⌋ + 1; parameter count = $K^2 \cdot C_{in} \cdot C_{out}$ (+ bias). Pooling, stride, dilation (wider receptive field at no extra cost). Batch normalization (stabilizes training; fold it into the convolution weights at inference). Activations: ReLU, GELU, SiLU.
- **Key architectures:** VGG (stacked 3×3 convolutions) → **ResNet** (residual connections fix vanishing gradients and allow 100+ layers) → Inception → **MobileNet** (depthwise-separable convolutions, about 8–9× fewer operations) → EfficientNet (compound scaling) → ConvNeXt (a modern CNN that rivals ViTs). Edge devices: MobileNetV3, EfficientNet-Lite, RegNet.
- **Classification:** softmax + cross-entropy; transfer learning (freeze the backbone, train the head, then fine-tune with a low learning rate). Class imbalance: weighted loss, **focal loss**, oversampling.
- **Object detection:**
  - Two-stage: Faster R-CNN (region proposal network + RoI heads). Accurate, slower.
  - One-stage: SSD, RetinaNet (focal loss), the **YOLO** family (v5–v11; anchor-free since v8). Real-time.
  - Transformer: DETR / RT-DETR (set prediction with Hungarian matching, no NMS).
  - Shared parts: anchors vs anchor-free; FPN (multi-scale features); **NMS** (sort by score, suppress boxes with IoU > 0.5, repeat); box regression losses (L1, GIoU/CIoU).
- **Segmentation:** semantic (per-pixel class: **U-Net** encoder–decoder with skip connections, the standard for industrial defects; DeepLab with atrous convolution); instance (Mask R-CNN; YOLO-seg); panoptic. Losses: cross-entropy, **Dice**/IoU loss for imbalanced masks. **SAM** (Segment Anything) does promptable zero-shot masks, which is useful for labelling.
- **Anomaly detection [Aurora deep-learning tools]:** defects are rare and varied, so train on **good parts only**.
  - Reconstruction-based: an autoencoder (or VAE/GAN) reconstructs the image; the anomaly score is the reconstruction error. Weakness: it can reconstruct defects too well.
  - Embedding-based: **PaDiM** (a Gaussian per patch position over pretrained CNN features; Mahalanobis distance), **PatchCore** (a memory bank of normal patch features with coreset subsampling; nearest-neighbour distance). Strong results on the MVTec AD benchmark.
  - Student–teacher and normalizing flows (FastFlow), EfficientAD (fast and accurate, suits edge devices).
  - Choose the threshold on a validation set with a few real defects; report image-level and pixel-level AUROC.

- **Training practice:** augmentation that matches real variation (rotation, flips if the part is symmetric, brightness, blur, noise; Mosaic/MixUp for detection); avoid augmentations that create unrealistic defects. Split train, validation and test by part or batch so near-duplicates don't leak. Learning-rate schedules (cosine, warmup), early stopping, weight decay. Few labels: transfer learning, **synthetic data**, self-supervised pre-training, active learning.
- **Vision Transformers (ViT):** split the image into 16×16 patches → linear embedding + position embedding → transformer encoder (multi-head self-attention, $O(n^2)$ in the number of tokens). Weaker built-in bias toward locality, so they need large-scale pre-training (DINOv2 features are excellent off the shelf). **Swin** uses windowed attention for linear cost and multi-scale features. For edge devices: costly attention, and int8 quantization is harder than for CNNs.
- **Vision-language models (VLMs):** an image encoder (CLIP-style, contrastive image–text training) + a projector + a large language model (LLaVA, Qwen-VL, GPT-4o). Uses in industry: zero-shot classification with text prompts, open-vocabulary detection (Grounding DINO, OWL-ViT), auto-labelling, explaining defects to operators. Limits: latency, hallucination, weak precise localization. Today they're for offline or server use, not the on-camera loop.
- **Classical or deep?** Classical: controlled conditions, measurement or gauging, need for explainability, tight cycle time, little data. Deep learning: high natural variation (organic textures, cosmetic defects), classification of complex appearance, OCR under poor print quality. Best practice is **hybrid**: classical to locate, align and measure; deep learning to classify or segment inside the aligned regions.

11. Deployment and optimization

- **Model export path:** PyTorch → **ONNX** → a hardware runtime: **TensorRT** (NVIDIA), **OpenVINO** (Intel CPU, GPU, NPU), ONNX Runtime (portable, with execution providers), TFLite / LiteRT (ARM, Android), or vendor NPU SDKs (Hailo, Qualcomm, Rockchip). Check the operator coverage of the target runtime early.
- **Quantization:** FP32 → FP16 (about 2× faster on a GPU, nearly no accuracy loss) → **INT8** (about 4× smaller, 2–4× faster). **Post-training quantization** needs a calibration set (100–1,000 representative images) to set scale and zero-point per tensor or per channel. **Quantization-aware training** recovers accuracy when PTQ loses too much. Sensitive layers (first, last, attention) can stay at FP16.
- **Other compression:** pruning (structured pruning, meaning whole channels, gives real speed-ups; unstructured pruning needs sparse hardware), **knowledge distillation** (a small student learns from a large teacher), operator fusion (Conv + BN + ReLU), lower input resolution or cropping to the region of interest.

- **Latency vs throughput:** batch size 1 for minimum latency on a production line; larger batches for throughput offline. Measure p50/p99 latency after warm-up, not the average of the first runs.
- **CPU optimization checklist:** profile first (perf, VTune), pick the algorithm, then memory (contiguous access, tiling for cache, no allocations in the hot path, aligned buffers), multi-threading (thread pool, split by tiles or rows, avoid false sharing), **SIMD** (SSE/AVX2/AVX-512 on x86, NEON on ARM; 32 uint8 pixels per AVX2 register; saturating arithmetic; handle the leftover tail), fixed-point arithmetic instead of floats.
- **GPU (CUDA/OpenCL) essentials:** one thread per pixel; warps of 32 threads; **coalesced** global memory access; shared memory for convolution tiles with halo; avoid branch divergence within a warp; occupancy. **PCIe transfer is often the bottleneck**, so keep data on the GPU across stages, use pinned memory and overlap copy with compute using streams. A small image may run faster on a CPU with SIMD.
- **Embedded / smart-camera constraints:** power envelope (often a few watts), thermal throttling, memory bandwidth, fixed-function blocks (ISP, hardware JPEG), DSP or NPU accelerators with limited operator support. Design for the weakest target and accelerate where the hardware allows.
- **Real-time pipeline design:** trigger → capture → process → decide → I/O. Run the stages as a pipeline on threads with bounded queues so frame n+1 is captured while frame n is processed. Use **zero-copy** buffers and preallocated memory pools. Keep timing deterministic: the worst case matters more than the average, because a late reject is a missed reject. Use a watchdog and a safe default (reject on timeout).
- **Productization:** a stable API, versioned models and configurations, reproducible builds, regression tests against golden outputs (bit-exact for integer code, a tolerance for float), performance benchmarks in CI, logging of failed images for retraining, and documentation of the design decisions.

12. Evaluation metrics

Metric	Definition	When it matters
Precision	TP / (TP + FP)	Cost of false alarms (good parts rejected)
Recall (sensitivity)	TP / (TP + FN)	Cost of escapes (bad parts shipped)
F1	2PR / (P + R)	One number when both matter equally

Metric	Definition	When it matters
False reject rate (FRR)	Good parts rejected / all good parts	Scrap cost and line yield; industrial "overkill"
False accept rate (FAR)	Bad parts accepted / all bad parts	Escapes to customers; often must be near 0
ROC / AUROC	TPR vs FPR across thresholds	Threshold-free comparison; anomaly detection
PR curve / AP	Precision vs recall; area under it	Better than ROC when positives are rare
IoU	Area of overlap / area of union	Box or mask quality; detection TP if IoU ⩾ 0.5
mAP@0.5, mAP@0.5:0.95	Mean AP across classes (and IoU thresholds, in the COCO metric)	Standard detection benchmark
Dice	2 × overlap / (size of A + size of B)	Segmentation, imbalanced masks
Reprojection error (RMS px)	Distance between projected and observed points	Calibration quality
Repeatability (3σ)	Spread of repeated measurements	Metrology and gauging
Latency p50 / p99	Median and tail processing time	Real-time budgets: design for p99

- **Threshold choice is a business decision:** set it from the relative cost of a false reject and a false accept, then verify it on a held-out set with real defects. In production, monitor the reject rate; a sudden change usually means **drift** (lighting, supplier, camera), not a real quality change.
- **Accuracy is misleading on imbalanced data:** with a 0.1% defect rate, "always good" scores 99.9% accuracy. Use recall at a fixed FRR, or the PR curve.
- **Your published numbers (never inflate):** F1 0.76 (audio pattern detection, JAES 2026), F1 0.78 (gesture fusion), 95% detection (SSIM method), 14 ms at 31.4% CPU on RPi4. Never ">90% F1".

13. Rapid-fire self-test

Cover the right-hand column and answer aloud in under 15 seconds each.

#	Question	Answer
1	Why blur before downsampling?	To prevent aliasing (Nyquist)
2	Gaussian kernel radius for σ?	About 3σ
3	Cost of a separable k×k filter?	O(2k) per pixel instead of O(k^2)
4	Box filter in O(1) per pixel?	Integral image or running sums
5	Best filter for salt-and-pepper noise?	Median
6	Edge-preserving smoothing?	Bilateral or guided filter
7	The five steps of Canny?	Smooth, gradient, NMS, double threshold, hysteresis
8	Harris response?	$\det(M) - k \cdot \mathrm{trace}(M)^2$
9	Why do corners track and edges don't?	Aperture problem: the structure tensor is rank-deficient on an edge
10	How are ORB descriptors matched?	Hamming distance (XOR + popcount)
11	Lowe's ratio test?	Accept if d1/d2 < 0.7–0.8
12	RANSAC minimal sample for a homography?	4 point pairs
13	What does Otsu maximize?	Between-class variance
14	Opening vs closing?	Opening removes small bright specks; closing fills small holes
15	Separate touching objects?	Distance transform + marker-controlled watershed
16	What is NCC invariant to?	Linear gain and offset in brightness
17	Why edge-based model finding over NCC?	Robust to lighting, occlusion and rotation
18	Sub-pixel peak formula?	Parabola fit through 3 samples
19	Homography DOF?	8
20	What do K, R and t hold?	Intrinsics; rotation and translation (extrinsics)
21	Two lens-distortion types?	Radial (k1–k3) and tangential (p1, p2)
22	Good calibration RMS?	Under about 0.3–0.5 px

#	Question	Answer
23	F vs E?	F uncalibrated (7 DOF); E calibrated, $E = K^T F K$ (5 DOF)
24	Stereo depth formula?	$Z = f \cdot B / d$; error grows with Z^2
25	Why Gray code plus phase shift?	Gray code is absolute but coarse; phase is fine but wrapped
26	Photometric stereo assumption?	Lambertian reflectance, known lights
27	ICP needs what?	A good initial alignment
28	Why U-Net for defects?	Skip connections keep fine spatial detail
29	PatchCore in one line?	Nearest-neighbour distance to a memory bank of normal patch features
30	INT8 post-training quantization needs?	A representative calibration set